Final Design and Analysis Report: *Aperture OMR*

Team 15

Graeme Brown Joel DeHoog Chad Glazier
Kaden Harris Tyson Livadney Nathan Worth

9 August 2026

Contents

1 Introduction and Setup **3**
1.1 Running the Service . . . 3

2 System Features **4**
2.1 Uploading Exams . . . 4
2.2 Marking Exams . . . 4
2.3 Inspecting Scans . . . 5
2.4 System Administration . . . 5
2.5 The Selling Point . . . 7

3 System Architecture **8**
3.1 Dataflow Diagrams . . . 8

4 Handover Guide **10**
4.1 Setting Up a Local Environment . . . 10
4.1.1 Code Generation . . . 11
4.2 Testing . . . 11
4.2.1 Current Testing Report . . . 12
4.3 Package Structure . . . 12
4.4 Dependencies . . . 13

5 Conclusion **14**

Preface

This document is part of Group 15's Final Design and Analysis Report for COSC 499. Our report is split into two parts: one describes the overall system and development process, without getting into the details of our OMR service, and the other focuses solely on the OMR service. This document is the latter.

This document describes the OMR as an independent service; it makes no mention of the broader web application that it was made for. To understand the place of this service in the complete system architecture, one should refer to the other document included in our submission.

1 Introduction and Setup

The *Aperture OMR* service, which we will refer to as "the OMR," is an Optical Mark Recognition (OMR) system exposed via an HTTP server. It can be run in a single Docker container, without relying on external services, and offers extremely fast marking operations on PDF scans of bubble-sheet exams. It requires some configuration before it can be used; namely, certain templates need to be created and uploaded before scans can be processed. This setup step makes the service ill-suited to one-off marking jobs, but it facilitates very efficient batch operations when frequently handling exams that share a configuration. It is recommended that the OMR is not used on its own but instead integrated into a broader app that handles the configuration programmatically.

1.1 Running the Service

In order to start the service, Docker[1] must first be installed. With the project downloaded and Docker running, the service can be started by navigating to the OMR's root directory (the directory containing the `go.mod` file) and running the following commands:

```
docker build --tag omr_server .
docker run -p 3000:3000 -v omr-data:/app/data --name omr omr_server
```

The build step may take up to a few minutes depending on the available hardware and internet connection. Once the container is running, logs should indicate that the server has started listening at `http://localhost:3000`. The API documentation can then be accessed through this address.

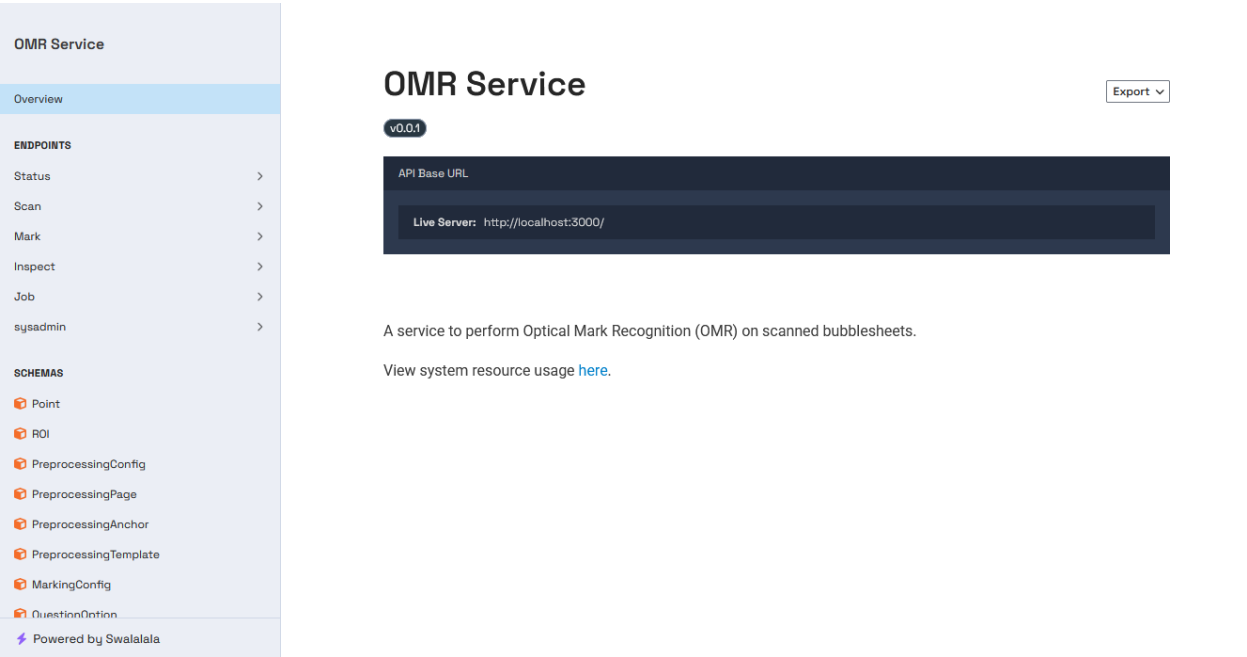


Figure 1: The API documentation page, viewable at `http://localhost:3000/`.

[1] At the time of writing, Docker can be installed from this page.

2 System Features

This section provides a walkthrough of the core features provided by the OMR. It references endpoints but it does not fully document them, leaving out the specifics of request bodies and possible error responses. For the full details, one should refer to the OMR's self-hosted API documentation. The documentation also includes small "try me" sections that can be used to manually make requests to the server; this functionality can be used to follow along with the guide if one so desires[2].

2.1 Uploading Exams

For uploading exams, there are two key endpoints:

1. `POST /template/preprocess` is used to upload a preprocessing template. The template informs the OMR about how a scanned exam *should* look, so that it knows how to correct pages that are misaligned, out of order, or skewed. Such a template must be uploaded before any scans.

2. `POST /scan/pdf` is used to upload one or more scanned exams in a single PDF. This is the primary way to upload scans. Once the scan is received by the OMR, it will render the PDF to an image format, preprocess the exams, and then store them. Stored scans can later be marked and viewed as images.

Rendering a large PDF is an expensive operation; large batches should be expected to take anywhere from a few seconds to over a minute depending on the hardware running the OMR. Fortunately, live progress updates can be retrieved if the upload is posted as an asynchronous job. The steps to do this are described below.

1. Instead of `POST /scan/pdf`, use `POST /job/scan/pdf` to upload the scans. This endpoint will immediately respond with a job ID for the request.

2. `GET /job` reports the current status of a job. The response will, among other things, include a "progress" value in the range of 0 to 1. A progress value of 1 indicates completion.

3. `GET /job/result` retrieves the results of a completed job. The response will be exactly the same as if the synchronous version of the request (`POST /scan/pdf`) was used. Job results are held by the server for up to 24 hours.

2.2 Marking Exams

After the exam scans have been uploaded, there are two endpoints used to obtain marks:

1. `POST /template/mark` is used to upload a marking template. A marking template includes information about which questions there are on the exam, where they can be found on a preprocessed page, and assigns each question a unique identifier.

2. `POST /mark` returns marks[3] for one or more exams that share a template. The marks that the OMR reports will contain a mapping of each exam's questions to its detected marked answers, labelling them

[2]Suitable sample data can be found in the source code, specifically in `internal/httpserver/handler/testdata`. This directory has everything needed to go through the full workflow.

[3]Semantically, this should be a `GET` request. However we need a proper request body so that the client can enumerate scan IDs without being bound by URL length limits.

with the question IDs matching those used by the template. Notably, the OMR does *not* have any kind of answer key, and it will not compute a score for the exams. It is solely responsible for detecting marks, not scoring them.

Unlike uploading PDFs, the marking operation itself is pretty fast, so it's not necessary to track it as an asynchronous job. However, network I/O may become a bottleneck–if you're requesting marks for, say, 200 scans at a time with 100+ questions per scan, you can expect responses to be tens of megabytes. At that size, the performance can be significantly improved by requesting that the data be compressed before transmission. The OMR will do this if the `Accept-Encoding` header is included with the request and is set to either `deflate` or `gzip`. Naturally, the response body will need to be decompressed by the client once it's received.

2.3 Inspecting Scans

Once a scan has been uploaded, you can use two endpoints to visually inspect it:

- `GET /image` is used to view an image of the full page for a scan. All you have to specify is the scan ID and the zero-based page index.
- `GET /image/snippet` is used to view a specific question on a scan. Naturally, you have to specify the scan ID and the marking template used to locate the question. The page index is inferred.

Both of these endpoints return an image directly and their parameters are specified in the query string. This means that you can treat them as static file URLs and include them in things like an HTML `<img/>` element's `src` attribute.

2.4 System Administration

Some endpoints were added to the OMR for the sake of resource monitoring and and debugging, which are listed below. If you want to view a summary of the data from these endpoints, you can visit the sysadmin dashboard while the OMR is running: `http://localhost:3000/dashboard.html`. Once you enter the admin key you should see a dashboard like the one in Figure 2. The admin key for the OMR is set when the server starts. You can configure it by passing it as an environment variable (`OMR_ADMIN_KEY`) when you start the service; it will default to `admin` if the variable is not set.

- `GET /system/logs` returns recent logs, the same as if you had viewed them in the regular terminal output.
- `GET /system/utilization` returns the current utilization of system resources (CPU, memory, and disk).
- `GET /system/cpu` returns the most recent CPU utilization samples[4].
- `GET /system/memory` returns the most recent memory utilization samples.
- `GET /jobs` returns a list of all recent job IDs. This endpoint requires an admin token.

[4]Samples for CPU and memory usage are collected once every second from the time the server starts up. The most recent 4096 samples are maintained at any given time (that's a little over an hour's worth of samples).

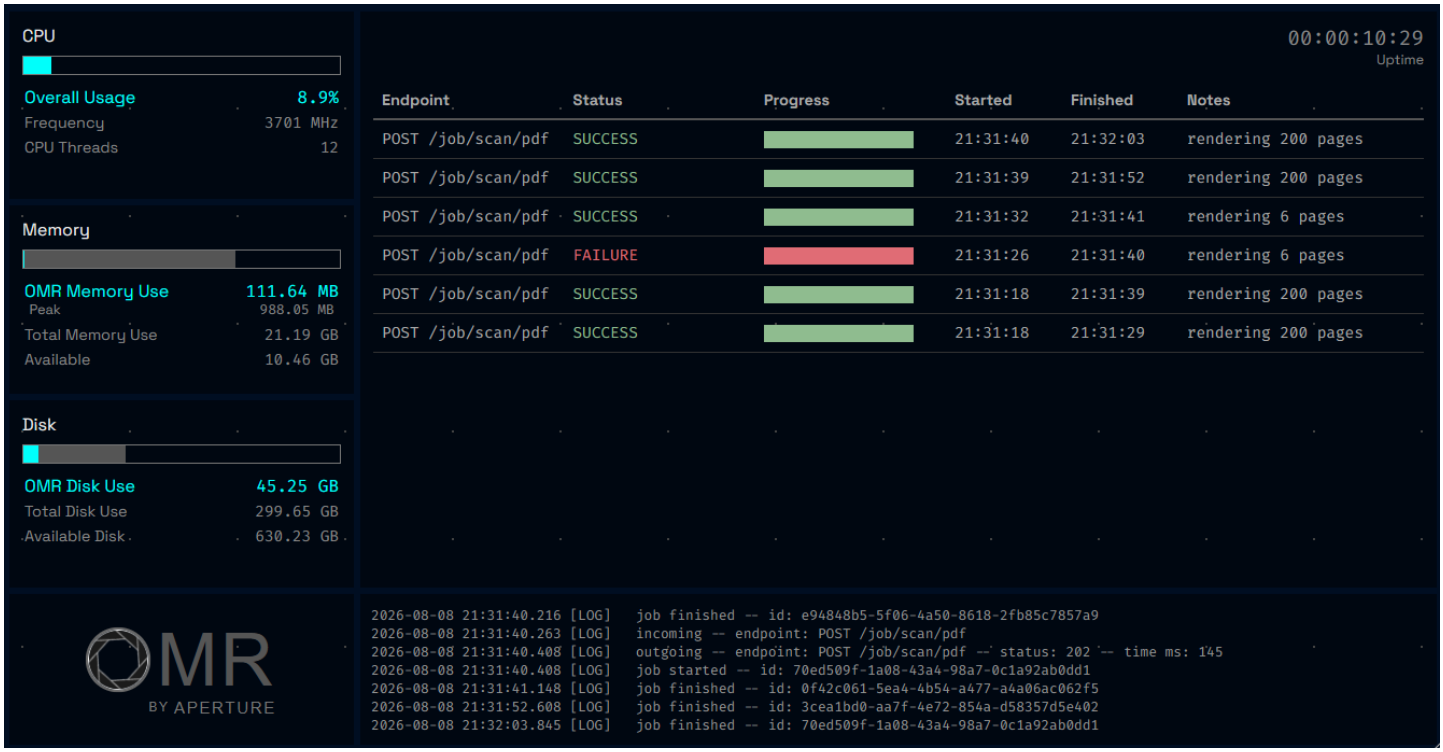


Figure 2: The admin dashboard at `http://localhost:3000/dashboard.html`. From here, system administrators can monitor large jobs, system logs, and resource usage.

2.5 The Selling Point

The primary value of the Aperture OMR, when compared to similar systems, is its performance. A great deal of effort was put into streamlining its processes to be mindful of memory while utilizing the full power of the CPU. A summary of its performance is given in Table 1.

Table 1: Performance benchmarks for the Aperture OMR service running on Windows 11. These operations were run against the OMR directly, via its documentation page. The hardware used is as follows: AMD Ryzen 5 7400F 6-Core Processor, 3600 MT/s RAM, and a Western Digital SSD connected via SATA. All response times are measured from the time the request was received until the time that the response was sent. The PDFs used were real scans filled in with pencil, with 100 questions on the first page and standard student information on the second.

Task	**Throughput**	**Description**
Scan Uploading and Preprocessing	20 pages per second	10 PDFs were uploaded to the server as separate jobs, each with 100 2-page exams. Each PDF was rendered at 250 DPI, preprocessed, and then stored. The total time it took complete all jobs was 99 seconds, with the fastest job completing in 9 seconds and the slowest completing in 11 seconds. Memory usage (RSS) peaked at 921 MB.
Scan Marking	102 pages per second	10 mark requests were sequentially sent to the server, each requesting marks for 100 2-page exams. The total time it took to complete all marking tasks was 19.7 seconds, with the fastest marking request completed in 1.9 seconds, and the slowest in 2.1 seconds. Memory usage (RSS) peaked at 214 MB.

3 System Architecture

The OMR uses a simple stack which is described in Table 2. A complete list of dependencies along with their versions is given in Table 3.

Table 2: The OMR's tech stack.

Component	Technology	Purpose
HTTP server	Go standard library	Provides the REST API and handles HTTP requests.
PDF rendering	Fitz / MuPDF	Renders PDF documents into images.
Image processing	OpenCV	Performs preprocessing and mark detection.
Database	SQLite	Stores templates and metadata for scans.
File storage	Local File System	Stores preprocessed scan images.

- Fitz/MuPDF was chosen for PDF rendering because it is well-established and reliable; the only caveat is that it has a high-level API and thus limits the control we have over rendering operations and the resources allocated to them.

- OpenCV was chosen for image processing because it is mature and extremely well-documented. However, it is not as specialized for document scans as some other alternatives. It's possible that other libraries could offer minor performance gains.

As mentioned before, the OMR runs in a single container. This is only possible because it uses SQLite instead of an external database, and it uses local file storage (on the Linux container) instead of an external object storage. While convenient, this also limits the scalability of the service by centralizing its data. Some consideration should be put into supporting alternative persistent storage methods if future work is done on the service.

3.1 Dataflow Diagrams

This section includes the dataflow diagrams for the core operations of the OMR.

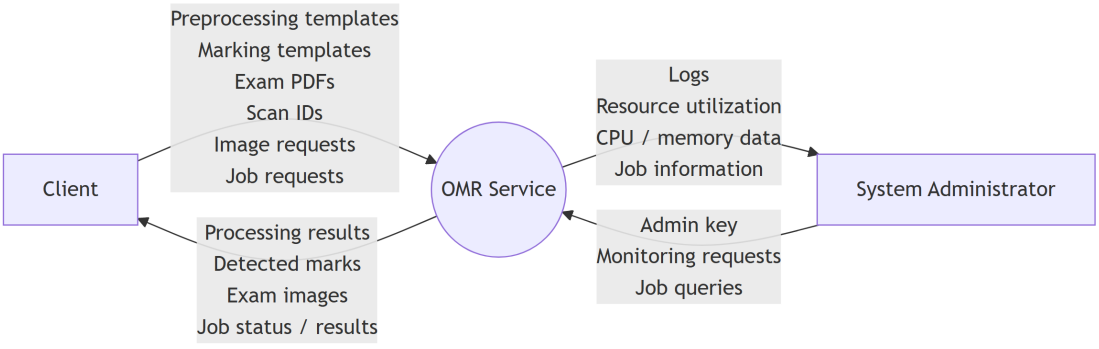


Figure 3: Level 0 Data Flow Diagram for the OMR.

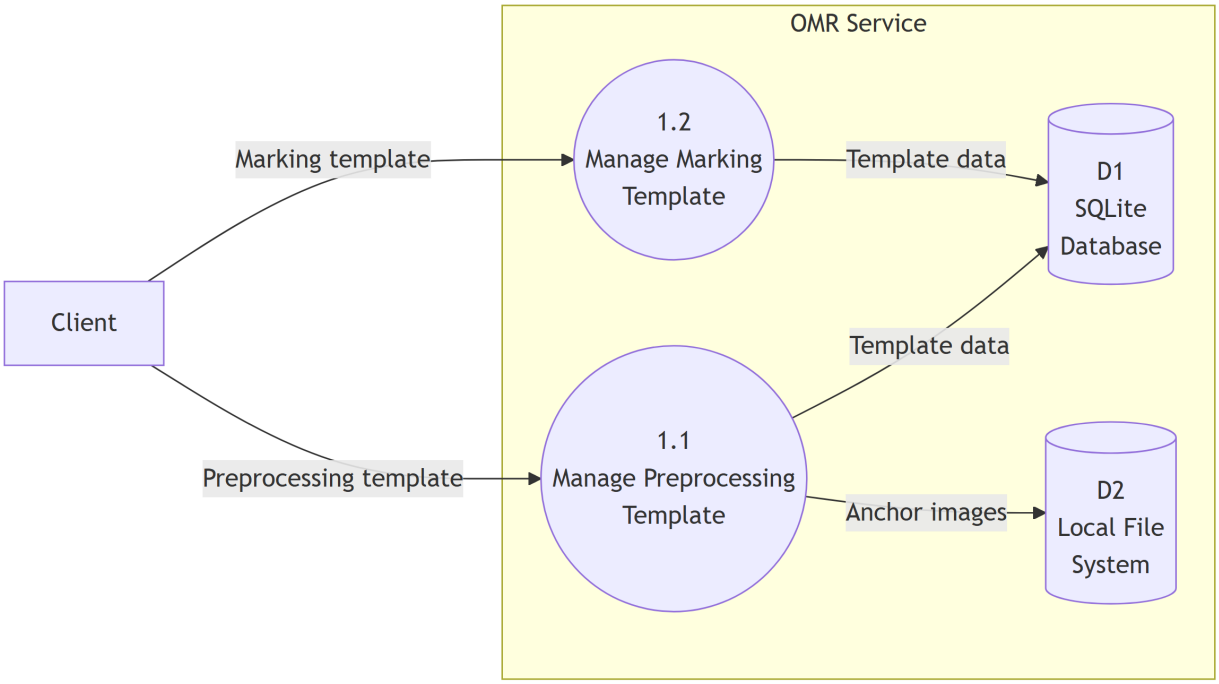


Figure 4: Level 1 Data Flow Diagram for uploading templates (both marking and preprocessing).

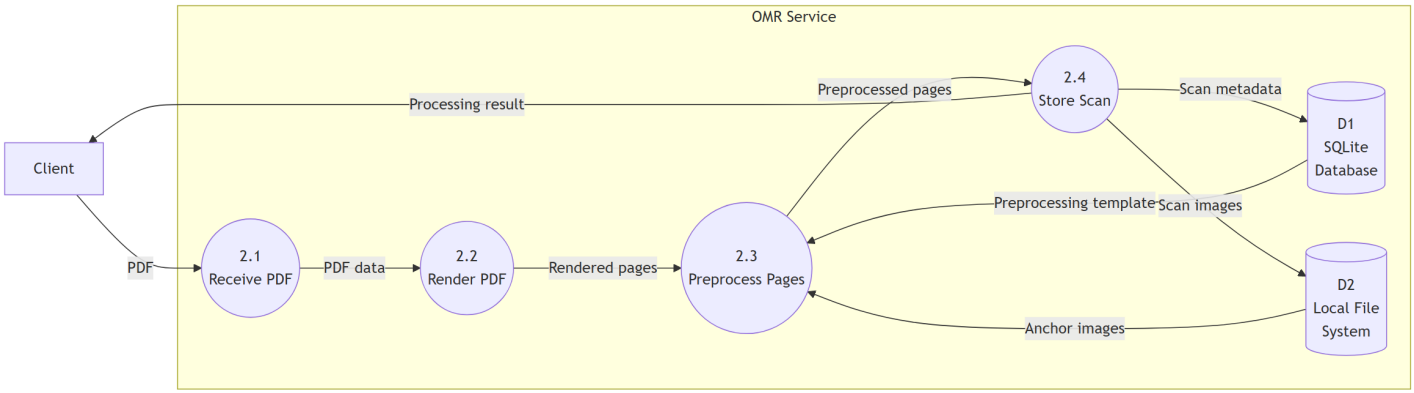


Figure 5: Level 1 Data Flow Diagram for a PDF upload.

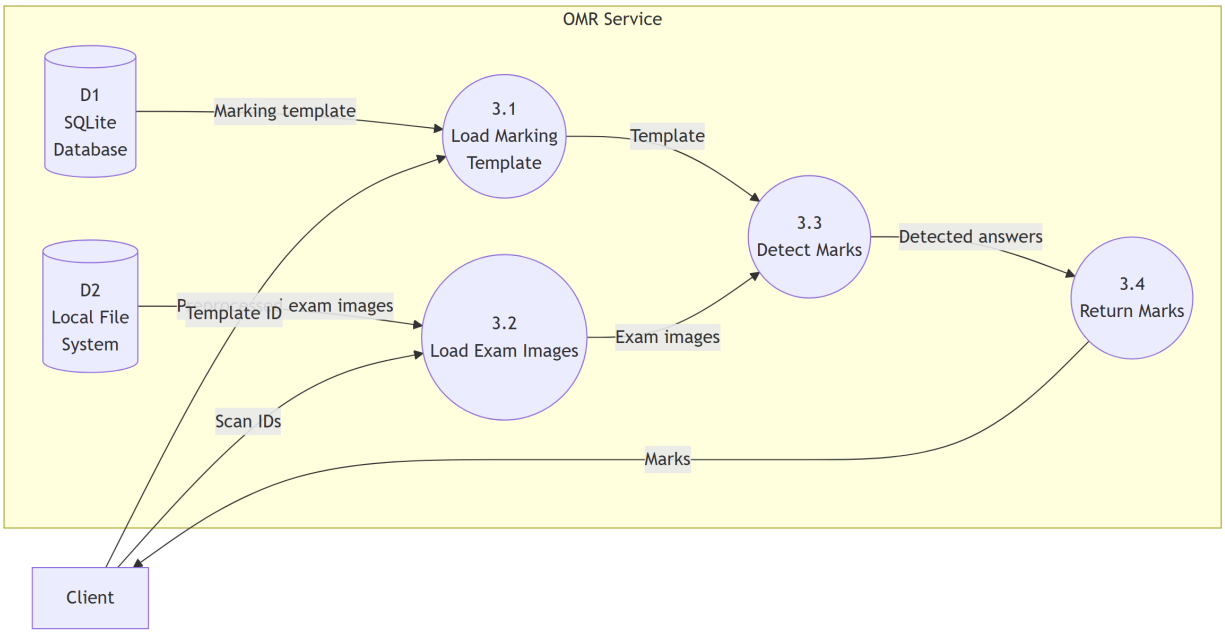


Figure 6: Level 1 Data Flow Diagram for requesting marks.

4 Handover Guide

This section explains how the Aperture OMR can be built upon from a developer's perspective. It begins by explaining how a local development environment can be set up, followed by an explanation of the tests, and finally provides an overview of the existing codebase to help navigate it.

4.1 Setting Up a Local Environment

In Section 1.1, the process for starting the service via Docker was explained. For application development, however, running the application locally is preferable. Fortunately there are only two things that need to be installed first:

- OpenCV, as a C++ library, is a build dependency. Clear and concise installation instructions for MacOS, Linux, and Windows can be found on this website.
- The Go Programming Language was used to write the program, so the compiler must also be installed.

Once both of those dependencies are installed, the program can be built by navigating to the top-level directory (the one with the `go.mod` file in it) and runing the following command:

```
# With Bash,
go build -o omr .
chmod u+x omr

# Or, with Windows,
```

```
go build -o omr.exe .
```

An executable should be produced in the working directory. The executable can be run to see some information about the available commands–notably, the `serve` command will start the HTTP server:

```
# Bash:
./omr serve

# Windows:
.\omr.exe serve
```

The documentation page can be found at `http://localhost:3000` while the server is running.

4.1.1 Code Generation

We use sqlc to generate Go functions that execute our SQL queries. sqlc can be installed via `go`:

```
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest
```

If the database needs to be modified, `internal/database/sqlc` should be opened and the `.sql` files changed directly. After finishing any modifications, the `sqlc generate` command can be run to re-generate the respective Go files. If repeatedly `cd`-ing in and out of the `sqlc` directory is tedious, one of the helper scripts for this can be used instead: `scripts/sqlc_generate.sh` for Bash, or `scripts/sqlc_generate.ps1` for Powershell.

4.2 Testing

The OMR's tests are all written for Go's testing tool, `go test`, which comes with the Go compiler. All tests can be run like so:

```
go test ./...
```

To get a per-package statement coverage report, the following script can be run.

```
# Bash:
chmod u+x ./scripts/coverage.sh
./scripts/coverage.sh

# Windows:
.\scripts\coverage.ps1
```

In keeping with Go conventions, all test files for a given package are kept alongside the files they test. For example, the functions defined in `ring.go` are tested in `ring_test.go`. For tests that require sample data (such as PDFs and large JSON files) accompanying `testdata/` directories are included.

Integration tests for the HTTP server are written as normal tests in `internal/httpserver/handler`. The Go standard library's `net/http/httptest` package is used to run HTTP requests on the handler functions.

Due to the way that our server's persistent storage is implemented[5], database and file storage operations can be trivially mocked during integration tests. See `image_test.go:15` as an example.

4.2.1 Current Testing Report

At the time of writing, the current coverage report is as follows (truncated for readability):

```
ok  ubco-team15/omr/internal/fs coverage: 81.9% of statements
ok  ubco-team15/omr/internal/httpserver/handler coverage: 80.9% of statements
ok  ubco-team15/omr/internal/httpserver/middleware  coverage: 51.0% of statements
ok  ubco-team15/omr/internal/marker     coverage: 89.5% of statements
ok  ubco-team15/omr/internal/pdf        coverage: 81.4% of statements
ok  ubco-team15/omr/internal/scanner    coverage: 79.8% of statements
ok  ubco-team15/omr/internal/sys        coverage: 82.1% of statements
[...]
total:                          (statements)  81.2%
```

The OMR has 169 distinct tests in total. Some of these tests are very long and could be refactored into distinct subtests.

4.3 Package Structure

The following is a list of all packages the OMR implements with a summary of their responsibilities.

- `cmd` contains the Cobra commands that serve as entrypoints to the program. The `rootCmd` command is used by the top-level `main.go` program.
- `internal/database` contains all database interactions.
- `internal/httpserver` contains the handler functions and middleware that make up the HTTP API for the service.
  - `dto` contains the data transfer objects (DTOs) for the server. Any complex object that will be sent or received from the server is defined there, along with relevant deserialization and validation functions.
  - `handler` includes the bulk of the HTTP server's logic.
  - `middleware` includes middleware functions.
- `internal/fs` exposes a simple interface for file storage, specifically for images and OpenCV matrices.
- `internal/pdf` handles PDF rendering.
- `internal/sys` centralizes logging and resource monitoring.

Other notable directories (that are not packages) are listed below:

- `api/` contains a single file, `openapi.yaml`. This file uses the OpenAPI specification to define the OMR's HTTP interface.

[5]See `internal/httpserver/resources.go`

- `pages/` contains static files that are served by the OMR. These files include the documentation and resource monitoring web pages.
- `scripts/` contains some short, useful scripts for managing the project. Scripts ending in `.sh` are for Bash, and those ending in `.ps1` are for Powershell (Windows).

4.4 Dependencies

Table 3: The OMR's direct dependencies. Indirect dependencies are not enumerated here, but can be found in the top-level `go.mod` file.

Go Package	Version	License	Purpose
gocv	0.43.0	Apache-2.0	Provides Go bindings for OpenCV, which we use for scan preprocessing and marking. The OpenCV C++ library is an indirect runtime dependency.
google/uuid	1.6.0	BSD-3-Clause	Generates UUIDs; primarily used to identify resources such as scans and jobs.
pdfcpu	0.13.0	Apache-2.0	Provides PDF utilities. Used to divide PDFs before parallel rendering.
pierrec/lz4	4.1.27	BSD-3-Clause	Provides LZ4 compression and decompression. Used for storing OpenCV matrices.
rs/cors	1.11.1	MIT	Provides CORS middleware for the HTTP server.
shirou/gopsutil	4.26.6	BSD-3-Clause	Provides access to information about the process and its resource usage.
spf13/cobra	1.10.2	Apache-2.0	Provides the CLI framework used to define and parse commands.
sync	0.21.0	BSD-3-Clause	Provides synchronization primitives not included in the Go standard library.
gen2brain/go-fitz	1.28.2	AGPL-3.0	Provides Go bindings for MuPDF, which we used to render PDFs. The MuPDF C library is an indirect runtime dependency.
modernc.org/sqlite	1.53.0	BSD-3-Clause	Provides a Go implementation of SQLite (version 3.53.0). Used for persistent storage of scan metadata and templates.

5 Conclusion

In this report, we have described the features and use cases for the Aperture OMR service, explained its architecture, and presented a brief hand-over guide for developers who wish to take on the project. The current state of the OMR is unpolished and significant work is required before it can be used in production. Nonetheless, the implementation presents a solid foundation and proof-of-concept for a high-performing alternative to existing marking systems.